

Name – Ankan Bhattacharya

Roll - 014

Problem Statement 1 - Create a multilevel class hierarchy (e.g., Vehicle → Car → SportsCar). Override a common method (e.g., display() or describe()) in each subclass, using the super keyword to invoke the immediate parent class's version before adding subclass-specific behaviour.

Program –

```
import java.util.Scanner;

class Vehicle {

    int wheel;

    Vehicle(int wheel) {

        this.wheel = wheel;

    }

    void display() {

        System.out.println("This is a Vehicle with " + wheel + " wheels");

    }

}

class Car extends Vehicle {

    String color;

    Car(int wheel, String color) {

        super(wheel);

        this.color = color;

    }

    @Override

    void display() {

        super.display();

        System.out.println("This is a Car with color " + color);

    }

}

class SportsCar extends Car {
```

Name – Ankan Bhattacharya

Roll - 014

```
int speed;

SportsCar(int wheel, String color, int speed) {

    super(wheel, color);

    this.speed = speed;

}

@Override

void display() {

    super.display();

    System.out.println("This is a Sports Car with speed " + speed);

}

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter no. of wheels");

        int inputWheels = sc.nextInt();

        sc.nextLine();

        System.out.println("Enter color");

        String color = sc.nextLine();

        System.out.println("Enter speed");

        int speed = sc.nextInt();

        SportsCar obj = new SportsCar(inputWheels, color, speed);

        obj.display();

        sc.close();

    }

}
```

Name – Ankan Bhattacharya

Roll - 014

Input –

Enter no. of wheels

4

Enter color

Red

Enter speed

200

Output –

This is a Vehicle with 4 wheels

This is a Car with color Red

This is a Sports Car with speed 200

Problem Statement 2 - Create an abstract class Shape with an abstract method area(). Derive subclasses Circle and Rectangle that override area() with their own implementation. Using an array/list of Shape references pointing to different subclass objects, call area() on each and demonstrate dynamic method dispatch (runtime polymorphism), explaining how the correct overridden method is resolved at run time.

Program –

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    private double radius;  
    Circle(double radius) {  
        this.radius = radius;  
    }  
    @Override  
    double area() {
```

Name – Ankan Bhattacharya

Roll - 014

```
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;

    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    double area() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(10, 4),
            new Circle(3)
        };

        for (Shape s : shapes) {
            System.out.println("Area = " + s.area());
        }
    }
}
```

Name – Ankan Bhattacharya
Roll - 014

Output –

Area = 78.53981633974483

Area = 40.0

Area = 28.274333882308138